

G05 Gate C — Marketplace Fulfilment Events

SEALED — OPEN ONLY WHEN EXPLICITLY ASSIGNED.

Independent assessment. No tutorial solution and no numeric pass threshold. Use only your completed M11-M14 learning, official tool help when necessary, and the supplied source data.

Source folder

05_GATE_C_SEALED/G05_GATE_C_SOURCE — dimension.csv, batch_facts.csv, delta_updates.csv, stream_events.csv, DATA_CONTRACT.txt.

Rules

Rule	Requirement
Attempt first	Save a genuine first attempt before opening Review.
Evidence	Record actual versions/commands/plans/counts/totals/offsets/checkpoints where executed.
No fake runtime	If Spark/Delta/Kafka cannot run, mark that competency runtime-pending; do not fabricate screenshots/output.
Preserve failure	A later retry never erases this attempt.
Pass	K1-K7 all demonstrated + no unresolved critical defect. Percentage is irrelevant.

K1 — Storage / Parquet

Profile the batch inputs and propose a Parquet/file-layout design. State schema, partition choice, small-file risk, compaction strategy, and one projection/pruning validation. If you can run Spark/PyArrow, save actual evidence; otherwise do not invent runtime proof.

K2 — Spark execution

Build a Spark DataFrame pipeline with explicit schema. Show at least one lazy transformation and action, inspect an execution plan, explain jobs/stages/tasks, and justify repartition vs coalesce for one changed output.

K3 — Spark correctness/performance

Join facts to the dimension safely. Prove one-side key uniqueness first, reconcile row count/value before and after the join, detect a deliberate fan-out test, inspect shuffle/skew, and explain when AQE/cache/broadcast/built-ins help or hurt. Include one transformation test and bounded driver inspection.

K4 — Delta transactional state

Create/describe a Delta current-state table. Apply the supplied update batch with MERGE/upsert semantics, preserve version/history evidence, prove idempotent rerun, explain schema enforcement/evolution, and demonstrate one time-travel or version comparison. Include SCD2 reasoning for one changing attribute.

K5 — Lakehouse engineering

Define Bronze/Silver/Gold grains/contracts for this domain. Explain what each layer accepts/rejects, how files are maintained/compacted, and diagnose one broken-layer scenario without deleting evidence.

K6 — Kafka state/delivery

Design topic/key/partition strategy for stream_events.csv. Demonstrate within-partition ordering, offsets, consumer-group capacity, replay behavior, idempotent sink boundary, and compatible vs breaking schema change.

K7 — Streaming integration/operations

Build or precisely defend the Python producer/consumer and Spark Structured Streaming + Kafka path. Preserve checkpoint identity, measure lag per partition, inject a crash/restart, prove duplicate/loss behavior, reconcile output, and give a 90-second incident defense backed by evidence.

Final defense

Explain the complete path from source files/events → distributed transform → transactional lakehouse state → streaming delivery/checkpoint → serving/recovery. Name at least three plausible silent-failure modes and the evidence that would expose each.